Large Language Models in Data Science

Week 2: Hugging Face & Transformers — Using Pretrained Models

Sebastian Mueller
Aix-Marseille Université
2026–2027

From Understanding a Model to Using One

Week 1: how tokens become contextual vectors and predictions.

Today: load a trained model, inspect its outputs, and measure whether it works for a task.

Lecture route

1. Choose a model and its files.
2. Run a pipeline, then open its steps.
3. Generate text and format a conversation.
4. Measure quality, runtime and memory.

Lab question

Which sentiment classifier would you use for movie reviews, and what evidence supports your choice?

Use DistilBERT and BERT for classification, then GPT-2 for generation. Small models let us inspect the mechanics.

Where Does the Model Run?

On your machine

- Download model files from the Hub.
- Python loads the model and runs it on your CPU or accelerator.
- You choose the input handling and can inspect the outputs.

Through a hosted API

- Send an input to a serving provider.
- The provider runs the model and returns the result.
- Access, limits and billing depend on the provider and account.

This lab starts locally. Hosted inference is an optional comparison.

What Does Hugging Face Provide?

In Week 1, a **Transformer** was a neural-network architecture.

Here, `transformers` is the name of a **Python library** for loading and using pretrained models, including BERT and GPT-2.

Component	**What we use it for**
Hub	Repositories of model files and datasets, with documentation and version history. Spaces host applications and demos.
`transformers`	Python code that loads a tokenizer and model, then runs it.
Datasets	Load, select and batch examples with their reference labels.

The Hub contains many kinds of models. The library supports multiple architectures, including some that do not use Transformer blocks.

Explore the Hub: Models and Data You Can Use

Start with a question: what would you like to build or analyse?

Models: try a capability

- DistilBERT sentiment
  Is a review positive or negative?
- Qwen3-0.6B
  Give an instruction and generate a response.
- Whisper
  Turn a speech recording into text.

Datasets: inspect real examples

- IMDB
  Movie reviews with sentiment labels: our lab's evaluation data.
- Banking77
  Customer banking queries in 77 categories: what is the customer asking for?

In the lab: explore Models and Datasets. Search a topic you care about; filter by task or language. Open one model's usage example and preview a few dataset rows.

What Do We Download When We Load a Model?

On the Hub, we choose a trained sentiment model: **DistilBERT sentiment**.

Its page points to files. Python downloads them to our machine:

Files we need	**What they provide**
Tokenizer files	The rules and vocabulary for turning a review into token IDs, as in Week 1.
Trained weights	The numbers learned during training. We load them instead of training the model ourselves.
Configuration	The settings needed to rebuild this model, including its size and output labels: `POSITIVE` and `NEGATIVE`.

A saved set of trained weights is called a **checkpoint**.

Load the tokenizer and model from the **same Hub model page**, so their files belong together.

Load the Tokenizer and Model from One Version

A saved version ID identifies the files from the previous slide: **tokenizer files, trained weights and configuration**.

The loading functions call this version `revision`.

```
from huggingface_hub import HfApi
from transformers import AutoTokenizer, AutoModelForSequenceClassification
name = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
# Look up the current version ID. Save it to reuse this version later.
version_id = HfApi().model_info(name).sha
# Load the tokenizer: text <-> token IDs.
tokenizer = AutoTokenizer.from_pretrained(name, revision=version_id)
# Load the classifier: configuration + trained weights.
model = AutoModelForSequenceClassification.from_pretrained(
    name, revision=version_id)
```

`tokenizer` prepares the text; it does not predict sentiment. `model` makes that prediction.

Neither loading call runs a prediction yet.

Model Files Can Also Include Python Code

In our example, the installed `transformers` library already knows how to run DistilBERT. We download its trained weights and settings.

Some Hub models also supply their own Python code. Loading the trained weights and running that code are **two separate choices**.

How are the weights stored?

`safetensors` is a file format for storing the trained weights. Reading this format does not execute Python code from the weight file.

May the model run extra code?

`trust_remote_code=True` allows Python code from the model's Hub repository to run on your machine.

Our DistilBERT example does not need it.

```
model = AutoModelForSequenceClassification.from_pretrained(
    name, revision=version_id,
    use_safetensors=True,  # Read weights in the safetensors format.
    trust_remote_code=False) # Do not run extra code from the Hub.
```

Prepare Python for the Lab

We will run the models locally in a Jupyter notebook. First install the Python libraries used by the course.

In a terminal, from the course repository folder:

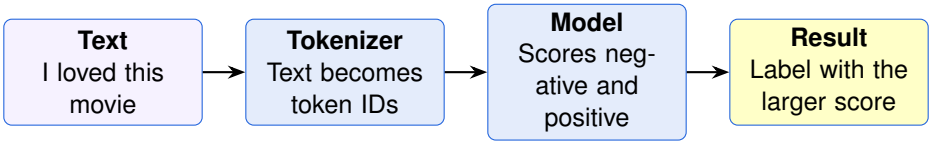
```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
jupyter lab labs
```

- `.venv` is a separate Python environment for this course's libraries.
- `requirements.txt` lists those libraries; `pip` installs them.
- Jupyter opens the lab notebooks. Select `week_2_huggingface_basics.ipynb`.

The first model download needs internet access and disk space. Our core models are public: no Hugging Face login is needed.

From a Review to a Prediction

We want to ask: is “I loved this movie” positive or negative?



`pipeline()` is a Python helper that connects these steps. We give it text and receive the result.

Using a trained model to make a prediction is called **inference**. Its weights stay fixed during this prediction.

Our First Call: Classify One Review

We specify the task and the trained model to use:

```
from transformers import pipeline
classifier = pipeline(
    "text-classification",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
    device="cpu")
result = classifier("I loved this movie")
print(result)
```

- `classifier` holds the pipeline: tokenizer, model and result handling.
- `device="cpu"` runs the calculation on the computer's main processor.
- The result contains a `label` and its `score`; inspect both in the lab.

Try "This movie was terrible" and "The movie lasts two hours." The model must choose positive or negative: it has no neutral label.

From Logits to Sentiment Probabilities

The model turns the review into a vector h. Its final layer calculates two scores:

1. Calculate the raw scores: logits

$$z_c = w_c^\top h + b_c$$

c is a class: negative or positive.
w_c: learned weight vector.
b_c: bias, a constant learned during training.

Logits can be negative. They are scores, not probabilities.

2. Convert the logits: softmax

$$p_c = \frac{e^{z_c}}{e^{z_{\text{neg}}} + e^{z_{\text{pos}}}}$$

Exponentiation makes both values positive. Dividing by their sum makes the probabilities add up to one.

The pipeline chooses the class with the larger p_c and returns that value as its `score`.

Illustrative example: $z_{\text{neg}} = -1$, $z_{\text{pos}} = 2$.

$$p_{\text{pos}} = \frac{e^2}{e^{-1} + e^2} \approx 0.953, \qquad p_{\text{neg}} \approx 0.047$$

Prediction: `POSITIVE`. A score of 0.953 does not establish a 95.3% chance that the sentence expresses positive sentiment.

Inside the Pipeline (1/2): Text to Raw Scores

Inspect the pipeline step by step, using the tokenizer and model loaded on slide 7.

$$\text{Text} \xrightarrow{\text{tokenizer}} \text{Token IDs} \xrightarrow{\text{model}} \text{Logits}$$

```
import torch
text = "I loved this movie" # 1. Supply the text.
inputs = tokenizer(text, return_tensors="pt") # 2. Tokenize.
print("Token IDs:", inputs["input_ids"])
model.eval() # Set training=False on the model and its layers.
with torch.inference_mode(): # 3. Run the trained model.
    logits = model(**inputs).logits # ** passes dictionary entries by name.
print("Raw scores:", logits)
```

`torch` is PyTorch, the library running the numerical calculations. `return_tensors="pt"` returns PyTorch arrays, called tensors.

`eval()` disables dropout: the random removal of activations used during training.

`inference_mode()` disables gradient tracking. Neither changes the learned weights.

Inside the Pipeline (2/2): Raw Scores to a Label

Continue with these logits: apply softmax, then select the label.

$$\text{Logits} \xrightarrow{\text{softmax}} \text{Probabilities} \xrightarrow{\text{largest value}} \text{Label}$$

```
# 4. Convert the two scores into class probabilities.
probabilities = logits.softmax(dim=-1)
for index, probability in enumerate(probabilities[0]):
    print(model.config.id2label[index], probability.item())
# 5. Select the label with the largest probability.
chosen_index = probabilities[0].argmax().item()
print("Prediction:", model.config.id2label[chosen_index])
```

- `probabilities[0]` selects our one review; its last dimension contains the two classes.
- `config.id2label` maps score positions to labels; `argmax()` finds the largest value's position.

Compare the label with the one returned by `classifier(text)`.

A Different Task: Fill a Missing Word

BERT can suggest a token at a marked position in the text:

```
fill_missing = pipeline(
    "fill-mask", model="google-bert/bert-base-uncased",
    device="cpu")
suggestions = fill_missing("Paris is the [MASK] of France.")
for suggestion in suggestions:
    print(suggestion["token_str"], suggestion["score"])
```

- `[MASK]` marks the position to fill. BERT uses the words on both sides as context.
- Each suggestion contains a candidate token and its model probability.
- Sentiment classification scores **two labels**; this task scores **30,522 tokens** in this BERT model's vocabulary (the tokenizer's fixed list of tokens).

The task determines what the model's output means. Later we will generate text one new token at a time.

Generate Text: Continue a Prompt

A **prompt** is the text we give the model. GPT-2 continues it by choosing a next token, appending it, and repeating — the loop from Week 1.

```
generator = pipeline(
    "text-generation", model="openai-community/gpt2",
    device="cpu")
result = generator(
    "A red toy robot",
    max_new_tokens=40, do_sample=False,
    return_full_text=False)
print(result[0]["generated_text"])
```

- `max_new_tokens=40`: stop after at most 40 new tokens.
- `do_sample=False`: choose the highest-probability token at each step.
- `return_full_text=False`: return only the new text, without the prompt.

GPT-2 was trained on web text to predict the next token from the preceding tokens.

Generation Settings: What Can We Change?

Compare with the previous call: now choose tokens randomly according to the model's probabilities. This is called **sampling**.

```
result = generator(
    "A red toy robot", max_new_tokens=40,
    do_sample=True, temperature=0.7, top_p=0.9, top_k=0,
    return_full_text=False)
```

- `temperature`: a lower positive value favours the most likely tokens more strongly; a higher value makes the probabilities more even.
- `top_p=0.9`: keep the most likely tokens until their probabilities total at least 0.9; sample within that set.
- `top_k=0`: disable an additional limit on the number of candidate tokens.

In the lab, change one setting at a time and repeat the call. Different wording is not evidence of better or more accurate answers.

A Conversation Must Be Formatted for the Model

GPT-2 continues plain text. A chat model is trained on conversations where markers identify **who is speaking** and where the next reply starts.

Messages we supply

- **user**: “Explain attention.”
- **assistant**: the model’s reply.

A message contains a **role** (the speaker) and **content** (their text).

The model receives one sequence

Illustrative format

[user starts] Explain attention.
[user ends] [assistant starts]

Where Do the Calculations Run?

The model's weights and its input arrays must be in memory on the device doing the calculation.

CPU: the computer's main processor

- Our examples use `device="cpu"`.
- This is a starting point that works without a separate graphics card.

GPU: a processor for parallel calculations

- NVIDIA GPUs: CUDA (`cuda`).
- Supported AMD GPUs: ROCm (also `cuda` in PyTorch).
- Compatible Macs: MPS (`mps`).
- It may speed up inference, but the model and inputs must fit in available memory.

Model weights take space, and the calculation needs additional working memory. A model that downloads successfully may still be too large to run.

In the lab: start on CPU. Try another available device only after the example works, and measure the difference.

Batching: Process Several Reviews Together

A **batch** is a group of inputs sent through the model in one call. To make a rectangular array, the tokenizer adds **padding** to shorter inputs.



The tokenizer also returns `attention_mask`: 1 for real input tokens, 0 for padding. It tells this classifier which positions to ignore.

More reviews per call may improve **throughput**: reviews processed per second. Larger batches also need more memory. Measure both.

The Lab: Use a Model and Defend Your Choice

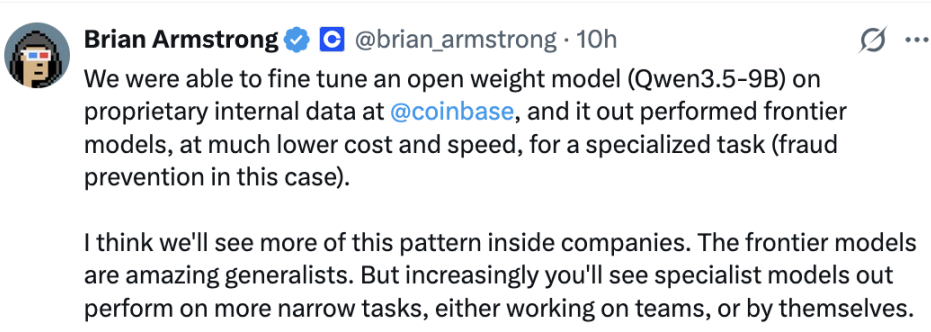
1. Run the sentiment pipeline. Try positive, negative and neutral reviews.
2. Run BERT's missing-token example and GPT-2's continuation. Explain how their outputs differ.
3. Open the sentiment pipeline: inspect token IDs, raw scores, probabilities and labels.
4. Compare batch sizes, then compare the two sentiment classifiers on the same IMDB sample.

Explain your choice: which model would you use, for which task, and what do its errors and measured runtime tell you?

Collaborate and use tools. Chat models, sentence similarity, precision experiments and hosted calls are optional extensions after the core comparison.

Outlook: Fine-Tuning for a Specific Task

Fine-tuning: continue training a pretrained model on task-specific data.



Brian Armstrong; screenshot: 12 Oct 2026. Coinbase write-up, 8 Oct 2026: company-reported fraud results.

Prompting changes the input; fine-tuning changes model parameters.
Test whether the adapted model improves results on examples **not used for training**.

Further Reading

- Pipeline tutorial
  Start with a task, model and input; then explore supported tasks.
- Hugging Face course: Behind the pipeline
  Walk through input preparation, model scores and result handling.
- Chat templates
  See how messages become the token sequence a chat model expects.
- Hub downloads and versions
  Reuse exact model files and understand the local cache.